IntellMeet — Mentor Project Report

Project owner: Shubham Singh
Project: IntellMeet — AI Meeting and Collaboration Workspace
Report date: 1 October 2026
Project type: Full-stack web application

1. Project overview

IntellMeet is an AI-assisted meeting and collaboration workspace. It combines meeting scheduling, authenticated user access, private meeting rooms, browser-based video collaboration, chat, recordings, transcription, AI-generated meeting intelligence, Copilot file analysis, tasks, analytics, reports, profile management, and support in one dark-themed interface.

The project was developed as a complete frontend and backend application rather than a static UI demonstration. Frontend actions communicate with typed backend APIs, user-owned data is protected on the server, and important meeting, support, upload, and AI records are persisted in the database.

2. Work completed

Project planning and architecture

- Defined the IntellMeet product scope and enterprise meeting-workspace direction.
- Organized the application into frontend, backend, shared types, database schema, migrations, scripts, and deployment configuration.
- Established a React/Vite single-page frontend with an Express/tRPC backend.
- Added a managed production architecture with static frontend assets, a Node backend container, database access, object storage, AI services, speech services, health checks, and published routing.

Authentication and user access

- Added email/password registration and login.
- Passwords are hashed with bcrypt before storage.
- Added project-scoped HTTP-only JWT session cookies.

- Implemented current-session lookup, protected dashboard access, logout, wrong-password rejection, and account ownership checks.
- Added an authenticated profile page where users can update their display name.
- Protected meeting records, uploaded files, meeting intelligence, support requests, and private meeting rooms by verified account identity.

Meeting creation and management

- Added meeting creation with title, description, date, start time, duration, and future-time validation.
- The backend derives the meeting end time from the start time and duration.
- Each saved meeting receives a deterministic private room ID in the form `meeting--<id>`.
- Added meeting listing for the authenticated owner.
- Added lifecycle states: `scheduled`, `live`, `completed`, and `cancelled`.
- Added host-controlled meeting cancellation.
- Completed and cancelled meetings cannot be joined or cancelled again.
- Added server-validated joining through the `meetings.join` API.
- Added email-based teammate invitations. Invitations grant private room access after the meeting owner shares the room link manually. The application does not currently send email notifications.
- Preserved the `/demo` route for browser-local demo meetings without requiring an account.

Dashboard and navigation

- Implemented the main dashboard and protected workspace navigation.
- Added overview, meetings, calendar, workspace, tasks, analytics, reports, profile, support, login, registration, demo, and call views.
- Added recent meeting cards, status labels, join/details actions, meeting dialogs, calendar navigation, and visible meeting controls.
- Preserved the original dark IntellMeet visual direction and responsive layout.

Meeting room and collaboration

- Implemented the meeting-room route and room access states.
- Added camera and microphone permission handling.

- Added local media stream management.
- Added WebRTC peer-connection handling.
- Added offer/answer exchange and ICE candidate exchange through Socket.IO signaling.
- Added camera, microphone, audio, screen-sharing, leave, disconnect, and resource-cleanup behavior.
- Added private-room access checks for the meeting owner or an invited account.
- Server-derived participant names prevent identity spoofing.
- Cross-room signaling and chat forwarding are rejected.
- Added in-room chat UI, recording download, and meeting controls.
- TURN infrastructure and multi-network media reliability remain deployment-dependent areas for future production hardening.

Transcription and AI meeting intelligence

- Added recording/upload validation for WebM, MP3, WAV, OGG, and M4A files under the configured size limit.
- Stores meeting recordings and uploaded files using managed object storage.
- Sends supported recordings to managed speech-to-text services.
- Provides an editable transcript view.
- Generates AI meeting summaries, key decisions, and structured action items.
- Persists signed-in meeting intelligence in the database and reloads it for the meeting.
- Allows generated action items to be added to the Tasks view.

IntellMeet Copilot

- Added general-purpose AI answers rather than restricting Copilot to one command type.
- Copilot receives workspace context and can answer questions about meetings, tasks, analytics, reports, and supported workspace actions.
- Added safe actions for opening meetings, navigating views, creating meetings, and adding tasks.
- Added visible attachment controls and multi-file selection.
- Uploaded files are stored durably, with metadata and ownership recorded in the database.
- Supports text extraction and multimodal analysis for supported text, image, PDF, audio, and video formats through managed AI services.

- Added loading, error, pending, and attachment-removal states.

Profile and support

- Added an authenticated profile page with persisted display-name editing.
- Added a support center with FAQs and help content.
- Added a validated support request form.
- Support requests are stored as database records with category, name, email, message, owner, and status fields.

3. Technologies used

Frontend

- **React 19** — component-based user interface.
- **TypeScript** — strict typing across UI, API contracts, and shared models.
- **Vite** — frontend development server and production bundling.
- **Wouter** — lightweight client-side routing.
- **TanStack React Query / tRPC client** — typed API calls, caching, invalidation, and mutation states.
- **Lucide React** — interface icons.
- **Sonner** — toast notifications.
- **CSS** — custom responsive dark visual system and accessible control styling.

Backend

- **Node.js 22** — server runtime.
- **Express** — HTTP application server and production static serving.
- **tRPC** — typed backend procedures and frontend/backend contracts.
- **Socket.IO** — real-time room signaling and chat events.
- **WebRTC** — browser peer-to-peer audio/video communication.
- **bcrypt** — password hashing.
- **JWT** — signed application sessions.
- **Zod** — request validation and input schemas.

Database and persistence

- **MySQL/TiDB-compatible managed database** — persistent application records.
- **Drizzle ORM** — typed schema definitions, queries, and migrations.
- Database tables include:
  - `users`
  - `local_accounts`
  - `meetings`
  - `meeting_invitations`
  - `copilot_files`
  - `meeting_insights`
  - `support_tickets`
- The deployed version uses MySQL/TiDB-compatible storage through Drizzle. It does not use MongoDB in the current managed deployment.

AI, speech, and file services

- Managed LLM inference for Copilot and structured meeting intelligence.
- Managed speech-to-text for uploaded recordings.
- Managed object storage for Copilot files and meeting recordings.
- Signed storage URLs are used when the model needs temporary file access.

Deployment and operations

- Docker production container based on Node.js 22.
- pnpm dependency installation and reproducible lockfile.
- Static Vite output in `dist/public`.
- Node server bundle in `dist/index.js`.
- Health endpoint at `/api/health`.
- `/api/*` routes are sent to the backend container.
- Hashed `/assets/*` files use immutable caching.
- SPA page routes use static fallback with fresh HTML behavior.

- Runtime secrets are injected by the managed platform and are not stored in the source archive.

4. Main API surface

Area	Procedures / endpoints
Authentication	`auth.register`, `auth.login`, `auth.me`, `auth.logout`, `auth.updateProfile`
Meetings	`meetings.list`, `meetings.create`, `meetings.join`, `meetings.cancel`, `meetings.invite`
Meeting intelligence	`meetingIntelligence.get`, `meetingIntelligence.transcribe`, `meetingIntelligence.analyze`
Copilot	`copilot.upload`, `copilot.ask`
Support	`support.create`
System	`system.notifyOwner`
Health	`GET /api/health`
Real-time	Socket.IO at `/api/socket`

5. Security and validation decisions

- Account identity is derived from the verified server session rather than client-supplied owner IDs.
- Passwords are never stored in plain text.
- Session cookies are HTTP-only and project-scoped.
- Meeting cancellation requires the authenticated meeting owner.
- Private meeting joining requires the owner or a stored invitation for the authenticated account email.
- Cancelled and completed meetings are rejected by the join API and private-room signaling layer.
- Socket.IO signals are forwarded only between sockets in the same authorized room.

- Participant names for private rooms come from the verified account session.
- Uploaded file ownership and metadata are stored server-side.
- API responses use private/no-store behavior where appropriate.
- No production secrets are included in the ZIP archive.

6. Testing and verification

The current release was verified with:

- `pnpm check` — TypeScript validation passed.
- `pnpm test -- --run` — **17 tests passed across 6 test files**.
- `pnpm build` — Vite frontend and Node backend production compilation passed.
- Database migration generation and application through `0007_meeting_lifecycle.sql`.
- Authentication smoke checks for registration, login, session lookup, wrong-password rejection, and logout.
- Meeting smoke checks for authenticated creation, ownership, listing, lifecycle status, invitations, and private access.
- Copilot upload and AI-answer smoke checks.
- Managed speech transcription smoke check using a WAV sample.
- Meeting-intelligence persistence and reload checks.
- Support request and profile update checks.
- Socket.IO checks for anonymous denial, uninvited-account denial, invited-account admission, verified participant identity, and cross-room signaling rejection.
- ZIP integrity verification with `unzip -t`.

7. Current limitations and future improvements

The project is a substantial functional full-stack application and academic/project submission, but it is not presented as a globally scaled enterprise conferencing platform. Future production hardening would include a TURN service for restrictive networks, multi-device WebRTC testing across network types, moderation and retention policies, outgoing invitation email delivery, persistent multi-user workspace chat, account-scoped task boards, event-based analytics, and fully account-scoped reports.

The current demo analytics values and some workspace/task/report views remain interactive prototype behavior. The `/demo` route intentionally uses browser-local state. Meeting invitations

persist and authorize account access, but the app currently expects the owner to share the link manually instead of sending email.

8. Mentor summary

IntellMeet progressed from project scope and architecture into a working full-stack meeting platform. The completed implementation includes a React/Vite frontend, Node/Express/tRPC backend, authenticated accounts, database-backed meeting scheduling and lifecycle management, WebRTC and Socket.IO meeting rooms, recordings and transcription, AI summaries and action items, Copilot file analysis, support, profiles, invitations, testing, migrations, and managed deployment configuration. The final source archive contains the implementation, migrations, documentation, test scripts, and project reports without production secrets.

Updated source archive: `IntellMeet-Final-Published.zip`